

LAPORAN LEMBAR KERJA 3

Implementasi dan Eksekusi

Pengembangan *Part Counting Station* Terintegrasi untuk Verifikasi Kuantitas Komponen dalam Kemasan Transparan Berbasis YOLOv5



Anggota kelompok

Stephanie Gabriella Wijaya	235150401111032
Maulana Aryan Wicaksana Sabandar	235150201111056
Nadhif Rif'at Rasendriya	235150201111074
Zaka Aulia Nala Udhma	235150301111041
Davin Kenaz Widiananda Tappo	235150301111026
Aulia Permata Kumala	235150701111001

UNIVERSITAS BRAWIJAYA
FAKULTAS ILMU KOMPUTER
2026

A. Identitas Proyek

- **Judul Capstone** :Pengembangan *Part Counting Station* Terintegrasi untuk Verifikasi Kuantitas Komponen dalam Kemasan Transparan Berbasis YOLOv5
- **Topik**: Sistem Verifikasi Kuantitas Part dalam Kemasan Tertutup
- **Mitra / Studi Kasus** : PT Indonesia Epson Industry
- **Nama Tim Proyek** : PT bebas aja asal dapat A

Komposisi Tim

No	Nama	NIM	Prodi	Peran	Kontribusi Utama
1	Stephanie Gabriella Wijaya	23515040 1111032	Sistem Informasi	Business Process Analyst	- Menganalisis permasalahan operasional pada proses pengecekan kuantitas <i>part</i> di area produksi. - Merancang dan mendokumentasikan proses bisnis sistem, termasuk alur <i>check</i> kuantitas, alur <i>recheck</i>, dan proses eskalasi saat terjadi <i>discrepancy</i>. - Merancang konsep sistem audit dan pencatatan digital untuk mendukung <i>traceability</i> dan <i>monitoring</i> proses inspeksi. - Merancang struktur <i>dashboard</i> KPI untuk memantau performa proses verifikasi dan tingkat <i>mismatch part</i>. - Menyusun format laporan inspeksi dan laporan <i>discrepancy</i> untuk mendukung proses <i>monitoring</i> dan evaluasi operasional.

2	Maulana Aryan Wicaksana Sabandar	23515020 1111056	Teknik Informatika	AI/ML Engineer	- Mengelola dataset (<i>data collection, labeling, quality control labeling</i>). - Melakukan <i>preprocessing</i> dan <i>augmentation</i> data untuk meningkatkan variasi dan kualitas dataset. - Menyusun pembagian dataset ke dalam <i>train, validation</i>, dan <i>test set</i>. - Melakukan analisis <i>error</i> untuk mengidentifikasi pola kesalahan model seperti <i>undercount</i> atau <i>overcount</i>. - Mencari strategi model agar <i>robust</i> terhadap <i>overlapping part</i>.
3	Nadhif Rifat Rasendriya	23515020 1111074	Teknik Informatika	AI/ML Engineer	- Menentukan pendekatan metode <i>part counting</i> berbasis <i>object detection</i> menggunakan YOLOv5. - Melakukan komparasi <i>training model</i> antara YOLOv5 dan SSD MobileNet. - Melakukan <i>bounding box labelling</i> pada <i>dataset</i> (Gambar <i>Peer Spring</i>). - Melakukan proses pelatihan model serta optimasi <i>hyperparameter</i> untuk meningkatkan performa deteksi dan <i>counting</i>. - Mencari strategi untuk memastikan model <i>robust</i> terhadap pantulan cahaya (<i>glare</i>).

					- Menentukan metrik evaluasi model seperti MAE, precision, recall, dan akurasi klasifikasi OK/NG. - Melakukan evaluasi performa model pada berbagai kondisi data untuk memastikan reliabilitas sistem <i>counting</i>.
4	Zaka Aulia Nala Udhma	23515030 1111041	Teknik Komputer	Hardware Engineer	- Melakukan integrasi perangkat keras dengan sistem pengolahan citra untuk mendukung proses pengambilan dan pemrosesan gambar secara stabil. - Merancang dan mengimplementasikan jalur komunikasi data antara perangkat <i>edge</i> (<i>Raspberry Pi 5</i>) dengan <i>Backend API</i>. - Mengembangkan skrip kontrol untuk sistem pencahayaan <i>custom</i> LED menggunakan sinyal <i>Pulse Width Modulation</i> (PWM). - Melakukan pemantauan performa sistem (seperti <i>thermal management</i> dan <i>stability stress test</i>) pada stasiun inspeksi.
5	Davin Kenaz Widiananda Tappo	23515030 1111026	Teknik Komputer	Hardware Engineer	- Merancang konfigurasi kamera, lensa, jarak pengambilan gambar dan penempatan <i>load cell</i>.

					- Merancang sistem pencahayaan dan <i>fixture</i> untuk meminimalkan pantulan plastik dan meningkatkan kualitas citra objek. - Melakukan pengujian stabilitas pengambilan gambar untuk memastikan konsistensi hasil deteksi pada <i>counting station</i>. - Mengkalibrasi <i>load cell</i> pada <i>station</i> agar sesuai dengan standar yang ditentukan
6	Aulia Permata Kumala	235150701111001	Teknologi Informasi	Backend Engineer	- Mendesain dan mengembangkan <i>backend API</i> untuk menerima hasil <i>counting</i> dari model. - Mendesain <i>database</i> untuk penyimpanan hasil inspeksi (<i>timestamp</i>, hasil hitung, status OK/NG). - Mengimplementasikan autentikasi dan <i>logging</i>. - Menyediakan <i>endpoint</i> untuk <i>dashboard</i> dan laporan. - Mengembangkan fitur <i>export</i> laporan dan notifikasi <i>mismatch</i>.

B. Ringkasan Solusi

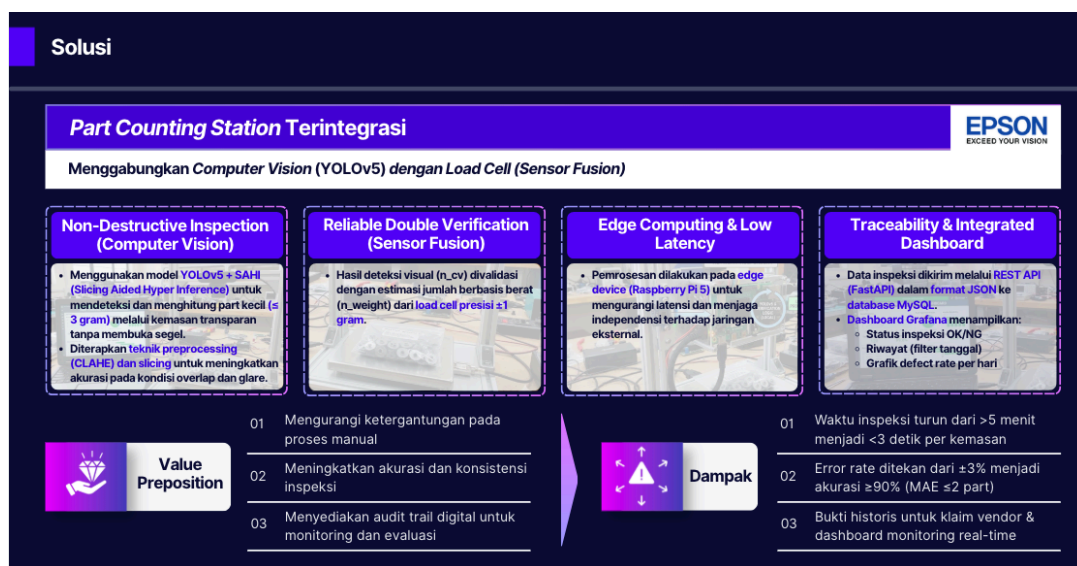
B.1 Masalah Yang Diselesaikan

Permasalahan utama yang diselesaikan adalah rendahnya efisiensi, akurasi, dan traceability pada proses verifikasi jumlah part di lini produksi PT Indonesia Epsilon Industry yang masih dilakukan secara manual. Proses inspeksi konvensional mengharuskan kemasan plastik atau tray dibuka untuk menghitung jumlah part satu per satu, sehingga memerlukan waktu lebih

lama, rentan terhadap *human error* akibat kelelahan visual, serta berisiko menimbulkan selisih jumlah part terutama ketika objek saling overlap atau terkena pantulan cahaya (*glare*) dari kemasan transparan. Selain itu, metode manual tidak menyediakan dokumentasi digital maupun audit trail yang memadai, sehingga apabila terjadi *mismatch* jumlah part yang baru terdeteksi pada tahap produksi lanjutan, kondisi tersebut dapat menyebabkan *downtime* produksi, kerugian operasional, hingga komplain dari customer

B.2 Solusi Yang Dibangun

Solusi yang dibangun berupa sistem otomatis *Part Counting Station* berbasis *Computer Vision* dan *sensor fusion* yang mampu melakukan verifikasi jumlah part secara cepat, akurat, dan non destruktif tanpa membuka kemasan plastik transparan. Solusi lebih detail dapat dilihat pada Gambar B.1.



Gambar B.1 Solusi Part Counting Station

B.3 Tujuan Sistem

B.3.1 Tujuan Umum

Mengembangkan dan mengimplementasikan *prototype* Part Counting Station Terintegrasi berbasis *edge computing* (Raspberry Pi) yang mampu melakukan verifikasi kuantitas *part* dalam kemasan transparan tertutup secara non-destruktif, otomatis, dan terintegrasi dengan sistem pencatatan digital (audit trail) serta dashboard monitoring *real-time*, sehingga meningkatkan akurasi, efisiensi, dan *traceability* proses inspeksi di PT Indonesia Epson Industry.

B.3.2 Tujuan Khusus

1. Membangun *backend* API (FastAPI) dan *database* (MySQL) untuk menerima data inspeksi, menyimpan histori, serta menyediakan akses data melalui endpoint RESTful.
2. Mengintegrasikan model YOLOv5 (dengan SAHI dan CLAHE) pada Raspberry Pi untuk deteksi *part real-time*.

3. Mengintegrasikan sensor load cell (HX711) pada Raspberry Pi untuk membaca berat total kemasan sebagai verifikasi ganda.
4. Menerapkan logika sensor fusion di *backend* FastAPI (bandingkan *n_cv* dan *n_weight*) untuk menentukan status OK/NG dengan *threshold* yang dapat dikonfigurasi.
5. Menyediakan pencatatan digital (audit trail) otomatis ke database MySQL lengkap dengan timestamp, operator, hasil deteksi, status, dan selisih.
6. Mengembangkan dashboard monitoring KPI menggunakan Grafana Cloud (via PDC Agent) yang menampilkan metrik *real-time* seperti total inspeksi, *defect rate*, OK/NG, serta tabel detail inspeksi yang dapat difilter.
7. Memastikan sistem dapat berjalan ≤ 3 detik per kemasan dan akurasi OK/NG $\geq 90\%$ setelah integrasi YOLO dan load cell.

C. Desain Sistem

C.1 Arsitektur Sistem

C.1.1 Arsitektur Sistem AI Module

Sistem ini menggunakan jenis arsitektur **Edge-Based Processing** yang diintegrasikan dengan *Pipeline Computer Vision* berlatensi rendah. Pemrosesan inferensi kecerdasan buatan (AI) dilakukan secara lokal (desentralisasi) di dalam stasiun kerja menggunakan *edge device*, sehingga mengeliminasi ketergantungan pada konektivitas *cloud* dan memastikan umpan balik verifikasi (OK/NG) dapat dieksekusi secara *real-time*.

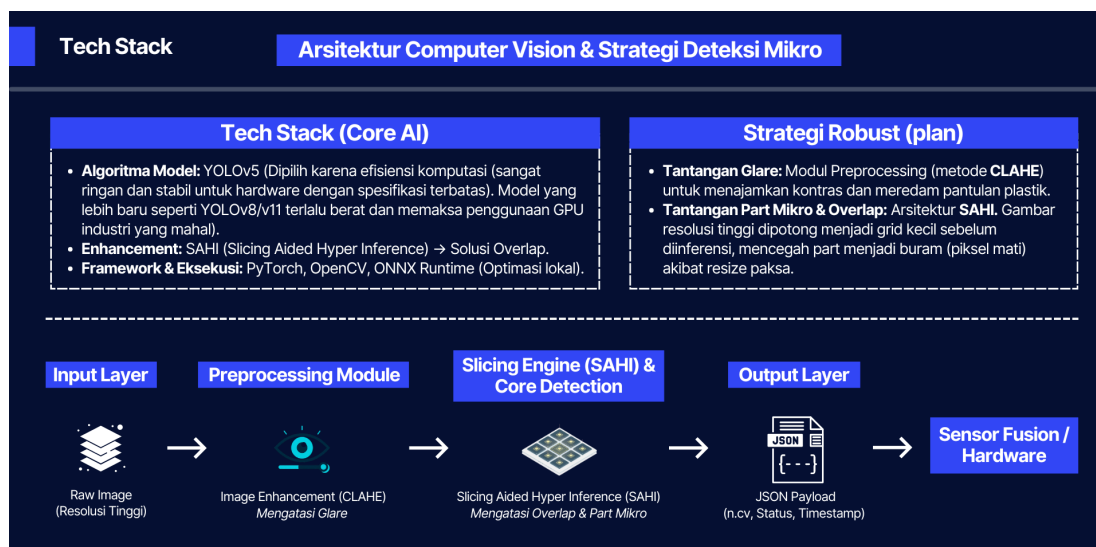
C.1.1.1 Komponen Utama (AI & Core Detection Pipeline)

Dari perspektif pemrosesan data visual, arsitektur sistem dibangun atas empat komponen utama yang bekerja secara sekuensial:

1. **Input Layer (Raw Image Acquisition):** Komponen penerima citra top-view beresolusi tinggi langsung dari perangkat kamera di stasiun inspeksi.
2. **Preprocessing Module:** Modul ini bertanggung jawab untuk melakukan *image enhancement*, secara spesifik menggunakan metode CLAHE (*Contrast Limited Adaptive Histogram Equalization*) untuk menajamkan kontras dan secara agresif meredam efek distorsi pantulan cahaya (*glare*) dari kemasan plastik transparan.
3. **Slicing Engine & Core Detection:** Komponen inti yang menjalankan teknik SAHI (*Slicing Aided Hyper Inference*) untuk memotong gambar resolusi tinggi menjadi *grid* berukuran kecil sebelum dimasukkan ke dalam model deteksi YOLOv5. Hal ini memastikan part berukuran mikro (di bawah 3 gram) yang saling bertumpuk (*overlap*) tidak mengalami distorsi piksel saat proses *resize* dan dapat terdeteksi secara akurat.
4. **Output Layer (JSON Payload Generator):** Modul ekstraksi yang mengonversi total *bounding box* hasil deteksi menjadi kuantitas part aktual (*n_cv*). Data ini kemudian dibungkus dalam format JSON (berisi *n_cv*, status, dan *timestamp*) untuk dikirimkan ke modul *Sensor Fusion* dan Backend API.

C.1.1.2 Teknologi Yang Digunakan (AI)

- **Algoritma Model:** YOLOv5. Arsitektur ini dipertahankan sebagai model inti karena tingkat efisiensi komputasinya yang sangat adaptif terhadap *edge device* dengan spesifikasi terbatas, serta terbukti menghasilkan tingkat akurasi (mAP50) yang jauh lebih superior dalam mendeteksi tumpukan part kecil dibandingkan arsitektur lain yang diuji.
- **Deteksi Objek Mikro (Enhancement):** SAHI (*Slicing Aided Hyper Inference*).
- **Framework & Manipulasi Citra:** PyTorch (sebagai *environment* utama pelatihan) dan OpenCV (untuk pra-pemrosesan citra numerik).
- **Deployment & Optimasi Lokal:** ONNX Runtime. Digunakan untuk mereduksi dan mengekspor bobot model agar dapat dieksekusi dengan kecepatan tinggi pada prosesor arsitektur ARM di *edge device*.



Gambar C.1 Diagram Arsitektur Tech Stack Core AI

C.1.2 Arsitektur Sistem Hardware Module

Arsitektur perangkat keras pada *Part Counting Station* dirancang menggunakan pendekatan **Edge-Heavy Architecture**, di mana seluruh proses pengambilan data, pengolahan citra (*inference*), dan logika keputusan dilakukan sepenuhnya secara lokal pada stasiun kerja. Hal ini bertujuan untuk memenuhi kebutuhan non-fungsional berupa latensi rendah (< 3 detik). Sistem dibagi menjadi tiga lapisan operasional:

1. **Lapisan Sensing (Sensor Layer):** Terdiri dari **USB Webcam** sebagai sensor visual untuk menangkap citra objek dan **Load Cell** yang terintegrasi dengan modul **HX711** untuk menangkap data berat fisik.
2. **Lapisan Komputasi Edge (Edge Processing Layer):** Berpusat pada **Raspberry Pi 5** yang berfungsi sebagai unit pemroses data mentah dari kedua sensor, menjalankan model **YOLOv5**, serta melakukan kalkulasi selisih (*discrepancy*).

3. **Lapisan Output & Komunikasi:** Mencakup **LCD Monitor** untuk memberikan umpan balik instan kepada operator dan modul **WiFi/LAN** untuk mengirimkan *payload* data (*inspection_id*, *quantity_cv*, *quantity_loadcell*, dll.) ke *backend*.

C.1.2.1 Komponen Utama (Hardware)

Berikut adalah spesifikasi perangkat keras yang digunakan dalam tahap implementasi:

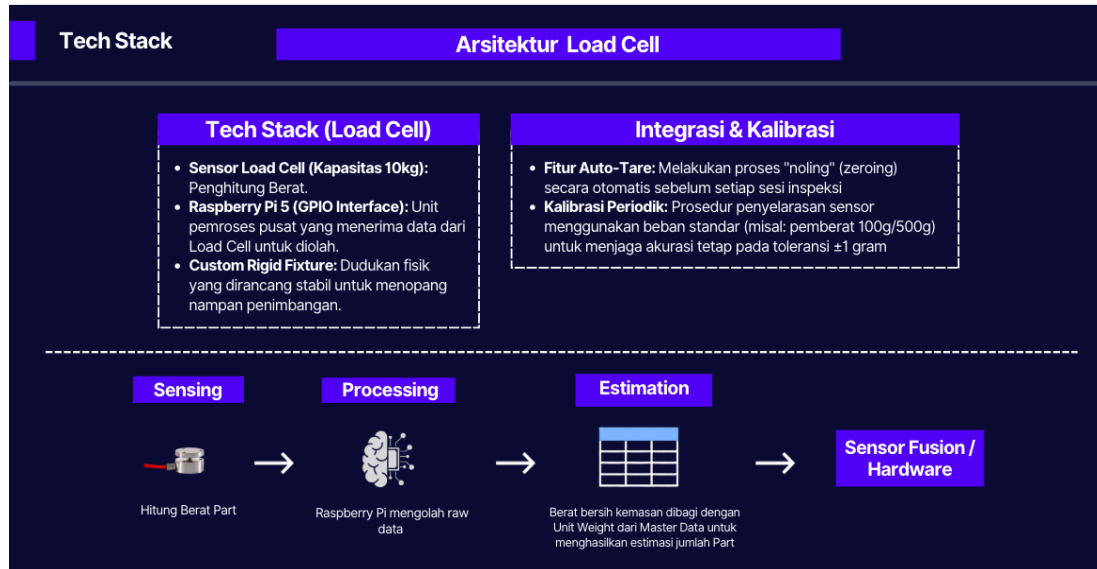
1. **Raspberry Pi 5 (8GB RAM):** Merupakan komponen utama penggerak sistem. Pemilihan versi 8GB memastikan kapasitas memori yang cukup untuk menjalankan *operating system* berbasis Linux sekaligus memproses inferensi AI tanpa *bottleneck*.
2. **USB Webcam Full HD (1080p):** Digunakan untuk mengambil gambar *top-view* dari kemasan *part*. Penggunaan webcam USB dipilih karena kemudahan integrasi, fleksibilitas kabel, dan stabilitas fokus yang lebih baik untuk objek mikro.
3. **Sensor Load Cell (Kapasitas 10kg):** Berfungsi untuk mendeteksi massa kemasan. Kapasitas 10kg memberikan rentang ukur yang luas namun tetap sensitif untuk mendeteksi perubahan berat pada *part* kecil (≤ 3 gram).
4. **Modul HX711 (ADC 24-bit):** Bertindak sebagai penguat sinyal (*amplifier*) dan konverter analog-ke-digital beresolusi tinggi untuk menjembatani data dari *load cell* ke pin GPIO Raspberry Pi.
5. **Monitor LCD 16x2 dengan I2C Adapter:** Perangkat *output* lokal yang menampilkan informasi *real-time* seperti *inspection_id*, jumlah hitungan, dan status OK/NG kepada operator di lapangan.
6. **Custom Station Fixture:** Struktur fisik tempat kamera dan timbangan diletakkan, dirancang untuk memastikan jarak fokus kamera ke objek tetap konsisten dan meminimalisir getaran eksternal pada timbangan.

C.1.2.2 Teknologi Yang Digunakan (Hardware)

Integrasi antar komponen didukung oleh beberapa teknologi pendukung perangkat keras berikut:

1. **Edge AI Inference:** Teknologi yang memungkinkan model *Deep Learning* (YOLOv5) berjalan secara mandiri di dalam Raspberry Pi 5. Hal ini menghilangkan ketergantungan pada koneksi internet untuk proses penghitungan visual, sehingga meningkatkan keandalan sistem di area produksi.
2. **Digital Signal Processing (Moving Average Filter):** Digunakan pada pemrosesan data dari sensor berat. Teknologi filter digital ini berfungsi untuk mereduksi *noise* atau fluktuasi angka akibat getaran mesin di sekitar area IQC, sehingga hasil *quantity_loadcell* menjadi sangat presisi.
3. **I2C Communication Protocol:** Protokol komunikasi serial yang digunakan untuk menghubungkan monitor LCD dengan Raspberry Pi menggunakan hanya dua kabel data (SDA & SCL), sehingga menyederhanakan manajemen pengabelan (*wiring*) di dalam *box* panel.

4. **Sensor Fusion Logic:** Teknologi integrasi data yang membandingkan masukan dari dua sensor berbeda. Logika ini memvalidasi hasil hitung kamera (n_{cv}) dan hasil estimasi berat (n_{weight}) terhadap nilai **Expected Quantity** yang diinputkan pengguna, menghasilkan verifikasi tiga arah (*Triple Check*) yang akurat.



Gambar C.2 Diagram Arsitektur Hardware

C.1.3 Arsitektur Sistem *Backend* Module

Modul *backend* dibangun menggunakan FastAPI sebagai *framework* REST API yang bertindak sebagai pusat koordinasi sistem. *Backend* menerima data hasil deteksi visual (n_{cv}) dari Raspberry Pi (YOLOv5) dan data berat ($weight_gram$) dari load cell, lalu menghitung estimasi kuantitas berdasarkan berat per unit *part* (n_{weight}). Logika sensor fusion membandingkan n_{cv} dan n_{weight} dengan ambang batas (*threshold*) untuk menentukan status OK/NG. Seluruh hasil inspeksi disimpan secara otomatis ke dalam *database* MySQL sebagai audit trail digital, lengkap dengan *timestamp*, operator, dan selisih. *Backend* juga menyediakan endpoint RESTful untuk mengakses histori inspeksi (`/inspections`), daftar part (`/parts`), serta mendukung autentikasi JWT dan *role-based access control* (RBAC). Dokumentasi API interaktif tersedia secara otomatis melalui Swagger UI pada `/docs`.

C.1.3.1 Komponen Utama (*Backend*)

Backend sistem dibangun dengan arsitektur modular menggunakan FastAPI sebagai *framework* utama. Komponen-komponen utama *backend* meliputi:

1. API Router yang menyediakan endpoint RESTful untuk komunikasi dengan perangkat edge (Raspberry Pi) dan dashboard. *Endpoint* utama meliputi:
 - POST `/api/v1/detection/inspect` untuk menerima data inspeksi (n_{cv} dari YOLO dan $weight_gram$ dari load cell), melakukan sensor fusion, dan menyimpan hasil.
 - GET `/api/v1/inspections` untuk mengambil histori inspeksi dengan filter (part, status, tanggal).

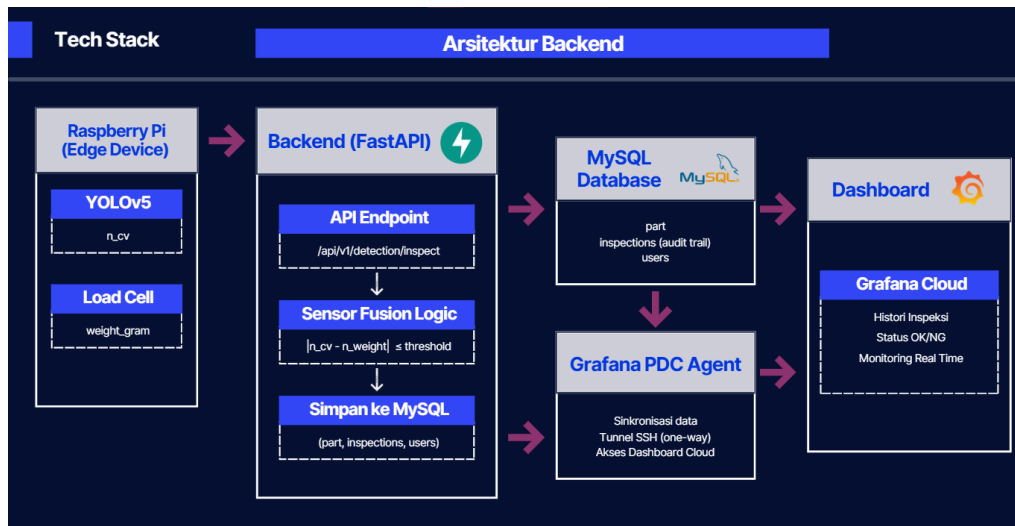
- GET /api/v1/parts untuk mengambil daftar part beserta parameter target_qty, threshold, dan weight_per_unit.
2. Database Layer (SQLAlchemy + MySQL) yang mengelola koneksi ke *database* part_counting_station dengan tabel part, inspections, dan users. Setiap inspeksi otomatis tersimpan sebagai audit trail digital.
 3. Logika Sensor Fusion yang menerapkan rumus $\text{difference} = |\text{n_cv} - \text{n_weight}|$ dan menentukan status OK jika $\text{difference} \leq \text{threshold}$, sebaliknya NG. Logika ini ditempatkan di app/services/sensor_fusion.py.
 4. Autentikasi & Otorisasi (Opsional) yang mendukung JWT dan role-based access (admin, supervisor, operator) yang dapat diaktifkan untuk melindungi endpoint sensitif.
 5. Dokumentasi API Otomatis yang tersedia melalui Swagger UI di /docs untuk memudahkan tim hardware dan dashboard melakukan integrasi.

Seluruh backend dideploy di server lokal (laptop) dan terhubung ke Grafana Cloud via PDC Agent. Komunikasi dengan Raspberry Pi menggunakan protokol HTTP dengan payload JSON, sedangkan koneksi ke dashboard cloud menggunakan PDC Agent sebagai tunnel SSH one-way tanpa membuka port firewall.

C.1.3.2 Teknologi Yang Digunakan (Backend)

- Web Framework: FastAPI. Framework Python modern dengan performa tinggi (async) dan dokumentasi API otomatis (Swagger UI). Dipilih karena memudahkan integrasi dengan tim hardware serta dashboard, dan sesuai untuk sistem real-time.
- ASGI Server: Uvicorn. Sebagai server web ringan yang mengimplementasikan standar ASGI, Uvicorn mendukung eksekusi kode asinkron secara efisien, sehingga latensi respons API tetap rendah meskipun beban request tinggi.
- Object-Relational Mapping (ORM): SQLAlchemy. ORM ini digunakan untuk memisahkan logika bisnis dari detail kueri SQL. Dengan SQLAlchemy, pengelolaan tabel part, inspections, dan users menjadi lebih terstruktur dan aman terhadap injeksi SQL, sekaligus memudahkan migrasi skema database jika diperlukan.
- MySQL Driver: PyMySQL. Driver murni Python yang menghubungkan SQLAlchemy ke server MySQL. Dipilih karena ringan, kompatibel lintas platform, dan mudah diintegrasikan.
- Manajemen Konfigurasi: python-dotenv. Digunakan untuk membaca variabel lingkungan dari file .env (seperti DATABASE_URL, SECRET_KEY). Memisahkan konfigurasi sensitif dari kode sumber, sesuai praktik keamanan standar. cryptography. Digunakan untuk mendukung metode autentikasi caching_sha2_password pada MySQL 8, sehingga memungkinkan koneksi dari PyMySQL tanpa mengubah konfigurasi database.
- Parsing Form Data: python-multipart. Diperlukan FastAPI untuk mem-parsing data dari form HTML, digunakan saat menerima payload inspeksi (part_id, n_cv, weight_gram) dari Raspberry Pi.

- Database Server: MySQL 8.4.3. Relational database yang menyimpan seluruh data inspeksi, part, dan user. Dipilih karena kestabilan, dukungan transaksi ACID, serta familiar di lingkungan industri dan proyek capstone.
- Koneksi ke Dashboard Cloud: Grafana PDC Agent. Jembatan tunnel SSH satu arah yang menghubungkan database lokal ke Grafana Cloud tanpa perlu membuka port firewall. Data historis tetap aman di lokal namun dashboard dapat diakses dari mana saja.



Gambar C.3 Diagram Arsitektur Backend

C.1.4 Arsitektur Sistem Dashboard Module

Modul *Dashboard* dibangun menggunakan *Grafana Cloud* sebagai platform visualisasi data untuk memantau performa sistem *Part Counting Station* secara *real-time*. *Dashboard* terhubung dengan *database* MySQL yang menyimpan seluruh hasil inspeksi dari backend FastAPI. Data inspeksi seperti jumlah total inspeksi, status OK/NG, *defect rate*, dan histori inspeksi divisualisasikan dalam bentuk panel KPI, gauge chart, bar chart, serta tabel monitoring interaktif. Modul ini mendukung *live monitoring* proses inspeksi, rekapitulasi hasil per hari, serta audit trail digital untuk kebutuhan analisis *Quality Control* (QC). Selain itu, *dashboard* dapat diakses oleh user berdasarkan role access tertentu melalui mekanisme autentikasi dan *role-based access control* (RBAC) yang terintegrasi dengan *backend system*.

C.1.4.1 Komponen Utama (Dashboard)

Dashboard sistem dibangun menggunakan *Grafana Cloud* sebagai platform visualisasi monitoring dan analisis data inspeksi secara *real-time*. Komponen-komponen utama dashboard meliputi:

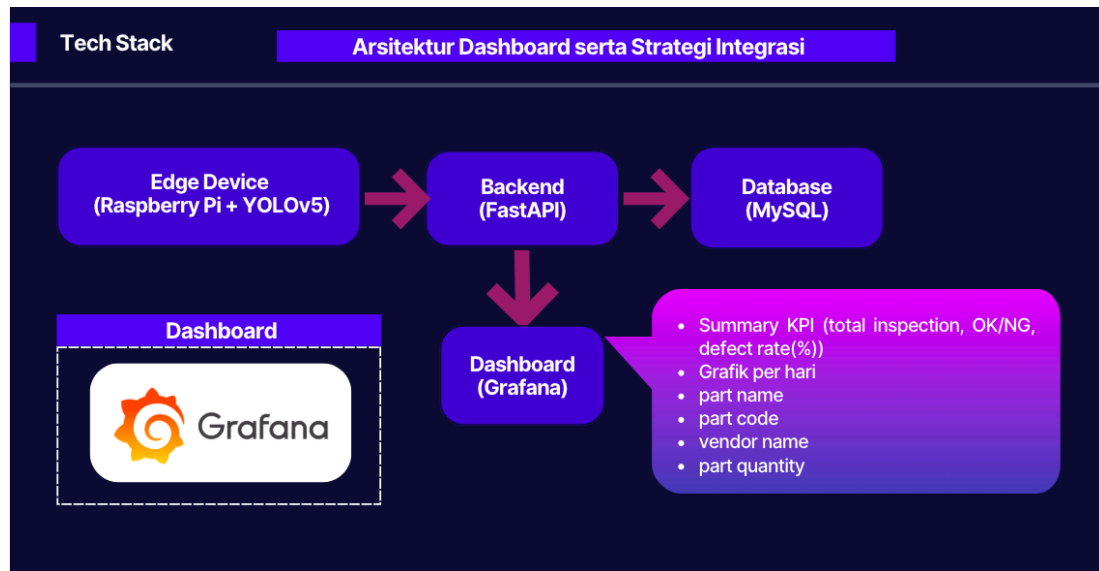
1. **Dashboard Visualization Panel:** Komponen utama yang menampilkan data hasil inspeksi dari backend dalam bentuk panel KPI, *gauge chart*, *bar chart*, dan tabel monitoring. Informasi yang divisualisasikan meliputi total inspeksi, total OK/NG, *defect rate*, serta histori inspeksi part.

2. **KPI Analytics Module:** Modul analitik yang mengolah data inspeksi menjadi indikator performa *quality control* seperti *defect rate*, rekam inspeksi per hari, jumlah *mismatch*, dan tren inspeksi harian untuk membantu proses evaluasi produksi.
3. **Inspection History & Audit Trail Module:** Komponen yang menampilkan histori inspeksi lengkap berdasarkan data yang tersimpan pada database backend. Data yang ditampilkan meliputi part name, operator, timestamp, kuantitas label, nilai *n_cv*, *n_weight*, difference, threshold, dan status inspeksi.
4. **Export & Reporting Module:** Modul yang menyediakan fitur export data inspeksi dan KPI dalam format laporan terstruktur untuk kebutuhan dokumentasi, evaluasi *quality control*, dan pelaporan operasional.

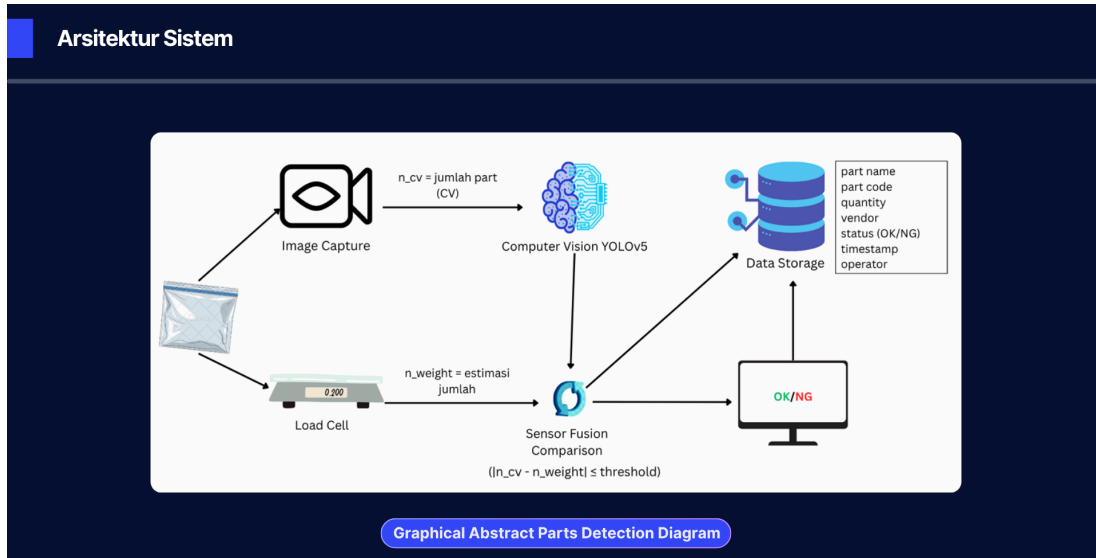
C.1.4.2 Teknologi Yang Digunakan (Dashboard)

Integrasi antar komponen dashboard dan sistem monitoring didukung oleh beberapa teknologi pendukung visualisasi dan konektivitas data berikut:

1. **Dashboard Platform (Grafana Cloud):** Platform dashboard berbasis cloud yang digunakan untuk menampilkan visualisasi KPI dan monitoring sistem Part Counting Station secara real-time. Dipilih karena mendukung berbagai jenis panel visualisasi dan mudah diintegrasikan dengan database monitoring.
2. **Data Visualization Engine (Grafana Panel):** Digunakan untuk membuat visualisasi data dalam bentuk gauge chart, bar chart, stat panel, dan table panel. Teknologi ini membantu pengguna memantau jumlah inspeksi, *defect rate*, status OK/NG, dan histori inspeksi secara interaktif.
3. **Data Source Integration (MySQL Data Source):** Digunakan untuk menghubungkan dashboard dengan database MySQL sehingga data inspeksi dapat divisualisasikan secara langsung pada dashboard KPI.



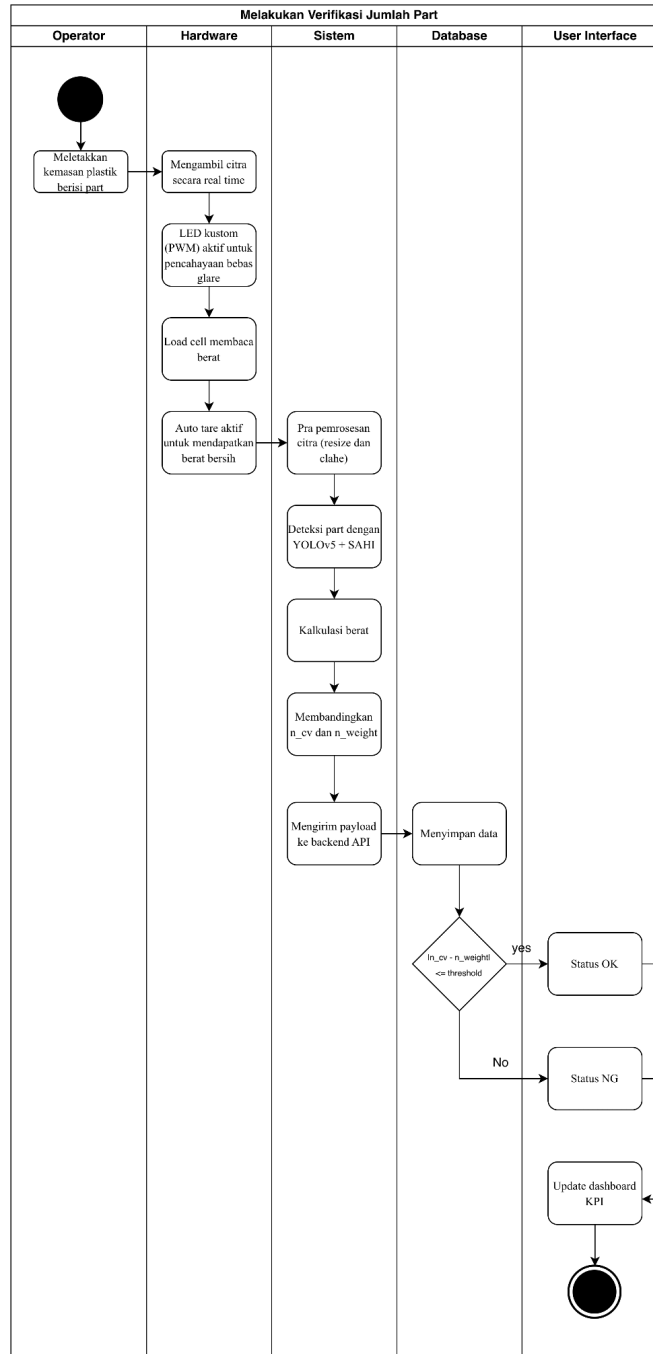
Gambar C.4 Diagram Arsitektur Dashboard



Gambar C.5 *Diagram Arsitektur Sistem*

C.2 Alur Sistem

Alur proses pada sistem *Part Counting Station* ini dirancang untuk menjalankan mekanisme verifikasi ganda (*dual verification*) secara otomatis dengan urutan sebagai berikut



Gambar C.6 Activity Diagram

C.3 Modul Sistem

Modul	Fungsi	Input	Output
Preprocessing & Image Enhancement	Melakukan pra-pemrosesan pada citra mentah untuk menajamkan kontras dan	Citra mentah (<i>Raw Image</i>)	Citra terproses (<i>Enhanced Image</i>) dengan kontras tinggi

	meredam distorsi pantulan cahaya (<i>glare</i>) dari kemasan plastik menggunakan metode CLAHE (<i>Contrast Limited Adaptive Histogram Equalization</i>).	resolusi tinggi dari kamera <i>top-view</i> .	yang siap untuk diinferensi.
Slicing Engine & Core Detection	Memotong citra menjadi <i>grid</i> berukuran lebih kecil menggunakan teknik SAHI (<i>Slicing Aided Hyper Inference</i>) dan mendeteksi objek <i>part</i> mikro yang saling bertumpuk (<i>overlap</i>) menggunakan arsitektur model YOLOv5.	Citra terproses (<i>Enhanced Image</i>).	Kumpulan data koordinat <i>bounding box</i> dari seluruh <i>part</i> yang berhasil dideteksi.
Counting & Payload Generator	Mengekstraksi dan menghitung total <i>bounding box</i> menjadi nilai kuantitas <i>part</i> aktual (<i>n_cv</i>), lalu membungkus data tersebut menjadi format terstruktur untuk dikirim ke sistem <i>Hardware/Backend</i> .	Kumpulan data koordinat <i>bounding box</i> .	<i>Payload</i> berformat JSON yang berisi kuantitas visual (<i>n_cv</i>), status awal, dan <i>timestamp</i> .
Dashboard KPI	Menampilkan visualisasi KPI secara real time berupa jumlah inspeksi, defect rate, jumlah OK/NG, tabel detail inspeksi untuk kebutuhan monitoring produksi dan quality control.	Data hasil inspeksi dan histori dari database/backend.	Dashboard visual interaktif berupa grafik KPI, statistik inspeksi, dan monitoring real-time.
API Gateway & Request Handler	Menerima dan memvalidasi request HTTP dari edge device (Raspberry Pi) serta dari dashboard. Melakukan routing ke endpoint yang sesuai (/detection/inspect, /inspections, /parts).	Payload JSON dari edge device (berisi <i>n_cv</i> , <i>weight_gram</i> , <i>part_id</i> , <i>operator_username</i>) atau parameter query dari dashboard.	Data tervalidasi yang diteruskan ke service layer, serta kode respons HTTP (200, 400, 404, 500).

Sensor Fusion & Logic Engine	Menjalankan logika verifikasi ganda: menghitung estimasi jumlah part dari berat ($n_weight = weight_gram / weight_per_unit$), menghitung selisih ($difference = n_cv - n_weight $), lalu membandingkan dengan threshold untuk menentukan status OK/NG.	n_cv (dari AI), $weight_gram$ (dari load cell), $weight_per_unit$ dan $threshold$ (dari database).	Status (OK/NG), nilai n_weight , $difference$, dan $threshold_used$.
Database Manager (ORM)	Mengelola seluruh operasi database: menyimpan hasil inspeksi, mengambil data part, menyimpan user, serta menyediakan query terfilter untuk dashboard. Menggunakan SQLAlchemy sebagai ORM.	Data inspeksi ($inspection_id$, $part_id$, $operator$, n_cv , n_weight , $difference$, $status$, $threshold_used$) atau parameter filter ($part_id$, $status$, tanggal).	Penyimpanan permanen ke tabel <code>inspections</code> , atau hasil query berupa objek Python yang siap diserialisasi ke JSON.
Audit Trail & Logging	Mencatat setiap inspeksi beserta timestamp, operator, dan hasil keputusan secara otomatis. Memastikan data tidak dapat diubah setelah tersimpan (immutable log).	Data inspeksi yang sudah diproses sensor fusion.	Entri baru di tabel <code>inspections</code> dengan <code>inspection_time</code> otomatis, serta log aktivitas sistem
Autentikasi & Otorisasi	Memverifikasi token JWT pada setiap request yang membutuhkan otorisasi (misal endpoint sensitif). Mengatur akses berdasarkan role (admin, supervisor, operator) menggunakan RBAC.	Header Authorization: Bearer <token> pada request HTTP.	Status autentikasi (valid/invalid), serta informasi user dan role yang digunakan untuk otorisasi.
Export & Reporting	Menyediakan endpoint untuk mengekspor data inspeksi dalam format CSV/Excel, baik untuk seluruh histori maupun berdasarkan filter tertentu.	Parameter filter (rentang waktu, part, status) dari request.	File CSV/Excel yang dapat diunduh pengguna, berisi data inspeksi sesuai filter.
Embedded Processing	Melakukan processing pada Embedded system dengan	Citra objek, Berat objek	Parameter hasil deteksi computer vision dan

	menggabungkan process antara computer vision dan load cell		parameter deteksi berat
--	--	--	-------------------------

D. Perencanaan Proyek (Sub-Cpmk-4)

D.1 Timeline Sederhana

Minggu	Aktivitas	Output
1-4	Perencanaan, analisis kebutuhan, desain sistem, penyusunan LK-1 & LK-2	LK-1 & LK 2 selesai
5-8	Eksplorasi tools: FastAPI, MySQL, YOLOv5, Grafana; riset hardware; setup environment	Tim paham tools, environment siap
9-10	Pengembangan backend FastAPI, pembuatan database MySQL, rakit hardware awal	Backend & database berfungsi, prototype hardware dasar
11-12	Integrasi edge device (RPi) ke backend, koneksi ke Grafana Cloud via PDC Agent, implementasi sensor fusion	End-to-end flow (RPi → backend → database → dashboard)
13-14	Pengembangan dashboard Grafana (KPI, tabel detail, filter) dan batch testing	Dashboard KPI real-time berfungsi
15-16	Perbaiki bug, persiapan presentasi & pameran	Sistem siap demo, Laporan final

D.2 Pembagian Tugas

Nama	Tugas
Aulia Permata Kumala	Backend Engineer: FastAPI, database MySQL, endpoint API, integrasi ke Grafana, dokumentasi API, audit trail
Stephanie Gabriella Wijaya	Business Process Analyst: analisis proses bisnis, desain dashboard KPI menggunakan Grafana, format laporan, eskalasi dan handling discrepancy
Maulana Aryan Wicaksana Sabandar	AI/ML Engineer: dataset, labeling, augmentasi, preprocessing (CLAHE), slicing (SAHI), training YOLOv5.
Nadhif Rifat Rasendriya	AI/ML Engineer: arsitektur & training YOLOv5, training SSD MobileNet, optimasi hyperparameter, komparasi model (YOLOv5 vs SSD MobileNet V2), testing & evaluasi model (akurasi, MAE).

Zaka Aulia Nala Udhma	Hardware Engineer: integrasi RPi, kamera, load cell, komunikasi edge-backend, kontrol LED PWM.
Davin Kenaz Widiananda Tappo	Hardware Engineer: konfigurasi kamera & lighting, kalibrasi load cell, stabilisasi pengambilan gambar.

D.3 Tools Yang Digunakan

Kategori	Tools
AI & Computer Vision	Google Colab (IPYNB), YOLOv5 (Ultralytics), OpenCV, PyTorch, SAHI, CLAHE, ONNX Runtime
Hardware (Raspberry Pi)	Raspberry Pi 5, Camera Module 3, Load Cell 5kg + HX711, LED PWM, RPi.GPIO, hx711py
Backend	FastAPI (Python), Uvicorn, SQLAlchemy, PyMySQL, python-dotenv, python-multipart, JWT
Frontend / Dashboard	Grafana Cloud (dengan PDC Agent untuk koneksi ke MySQL lokal)
Database	MySQL 8.4.3 (Laragon atau langsung via MySQL CLI)
Version Control	Git, GitHub
Development Tools	VS Code, Laragon (MySQL), Postman, Grafana, Google Colab, Roboflow
Jaringan	PDC Agent (Grafana)

E. Implementasi (Sub-Cpmk-5)

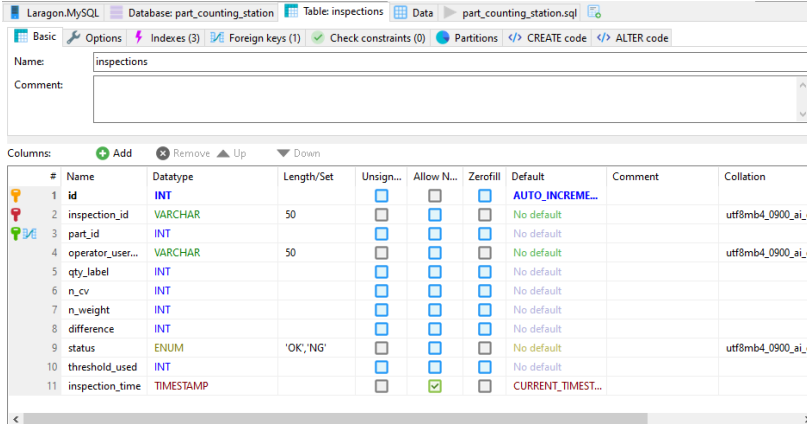
E.1 Fitur Yang Sudah Dibangun

Fitur	Deskripsi	Status (✓/✗)
Dataset Collection & Labeling	Mengumpulkan video part spring dalam kemasan plastik transparan dengan variasi glare, overlap, dan sudut pandang. Melakukan bounding box labeling menggunakan Roboflow dengan class "spring"	✓

Model Training & Testing using YOLOv5	Melatih model deteksi objek menggunakan arsitektur YOLOv5 pada platform Google Colab. Proses mencakup augmentasi data, pelatihan hingga 150 epochs, dan inferensi pada dataset uji (test set). Model berhasil mencapai akurasi sangat tinggi dengan nilai mAP50 99.1% dalam mendeteksi tumpukan part spring secara presisi.	✓
Model Comparison (YOLOv5 & SSD MobileNet V2)	Melakukan studi komparatif dengan melatih model alternatif (SSD MobileNet V2) menggunakan MediaPipe Model Maker. Hasil pengujian empiris membuktikan YOLOv5 jauh lebih superior dalam menangani objek mikro yang bertumpuk (<i>overlap</i>).	✓
Automated Counting & Data Extraction Script	Membangun dan memodifikasi skrip Python yang terintegrasi dengan modul deteksi YOLOv5 (detect.py). Skrip ini secara otomatis mengekstrak koordinat <i>bounding box</i> ke dalam format .txt dan mengonversinya menjadi teks kalkulasi numerik otomatis (jumlah <i>spring</i>) di layar secara <i>real-time</i> .	✓
Model Training (SSD MobileNet - Alternatif)	Melatih model SSD MobileNet sebagai model alternatif untuk perbandingan performa. Dilatih di lingkungan lokal (VS Code, Python 3.9 + TF 2.10.1) dengan 5000 steps, mencapai mAP@0.50 = 86.93%	✓
Dashboard KPI	Menampilkan visualisasi data inspeksi part secara real-time melalui dashboard dengan Grafana Cloud yang terhubung dengan database sistem. Dashboard digunakan untuk memantau performa proses <i>counting</i> dan <i>quality control</i> secara langsung.	✓
Database My SQL	Pembuatan database part_counting_station dengan tabel part, inspections, users, serta view inspection_details. Database terhubung ke backend FastAPI, menyimpan seluruh histori inspeksi sebagai audit trail.	✓
API Backend (FastAPI)	REST API dengan endpoint /api/v1/detection/inspect, /api/v1/inspections, /api/v1/parts. Dilengkapi dokumentasi Swagger UI, berjalan di server lokal dengan Uvicorn.	✓

Sensor Fusion Logic	Logika verifikasi ganda: $n_weight = weight_gram / weight_per_unit$, $difference = n_cv - n_weight $, status OK jika $difference \leq threshold$. Ditempatkan di app/services/sensor_fusion.py.	✓
Integrasi Grafana Cloud via PDC Agent	Pemasangan dan konfigurasi PDC Agent (tunnel SSH one-way) untuk menghubungkan MySQL lokal ke Grafana Cloud. Dashboard dapat menampilkan data real-time tanpa membuka port firewall.	✓
Integrasi Raspberry pi dengan Webcam dan Load Cell	Penggabungan antara hasil deteksi computer vision dengan Webcam dan Load Cell untuk menghasilkan parameter pendeteksi	✓

E.2 Bukti Implementasi

Fitur	Bukti Implementasi
Dashboard KPI	
Database My SQL	

Laragon.MySQL Database: part_counting_station Table: part Data part_counting_station.sql

Basic Options Indexes (2) Foreign keys (0) Check constraints (0) Partitions CREATE code ALTER code

Name: part

Comment:

Columns: Add Remove Up Down

#	Name	Datatype	Length/Set	Unsigned	Allow Null	Zerofill	Default	Comment	Collation
1	id	INT		☐	☐	☐	AUTO_INCREMENT		
2	part_code	VARCHAR	50	☐	☐	☐	No default		utf8mb4_0900_ai_ci
3	part_name	VARCHAR	100	☐	☐	☐	No default		utf8mb4_0900_ai_ci
4	weight_per_unit	DECIMAL	10,4	☐	☐	☐	No default		
5	target_qty	INT		☐	☐	☐	No default		
6	threshold	INT		☐	☒	☐	5		

Laragon.MySQL Database: part_counting_station Table: users Data part_counting_station.sql

Basic Options Indexes (2) Foreign keys (0) Check constraints (0) Partitions CREATE code ALTER code

Name: users

Comment:

Columns: Add Remove Up Down

#	Name	Datatype	Length/Set	Unsigned	Allow Null	Zerofill	Default	Comment	Collation
1	id	INT		☐	☐	☐	AUTO_INCREMENT		
2	username	VARCHAR	50	☐	☐	☐	No default		utf8mb4_0900_ai_ci
3	password_hash	VARCHAR	255	☐	☐	☐	No default		utf8mb4_0900_ai_ci
4	full_name	VARCHAR	100	☐	☐	☐	No default		utf8mb4_0900_ai_ci
5	role	VARCHAR	20	☐	☒	☐	NULL		utf8mb4_0900_ai_ci

Help Discard Save

Laragon.MySQL Database: part_counting_station View: inspection_details Data part_counting_station.sql

Options CREATE code

Name: inspection_details

Definer: root@localhost

SQL security: Definer

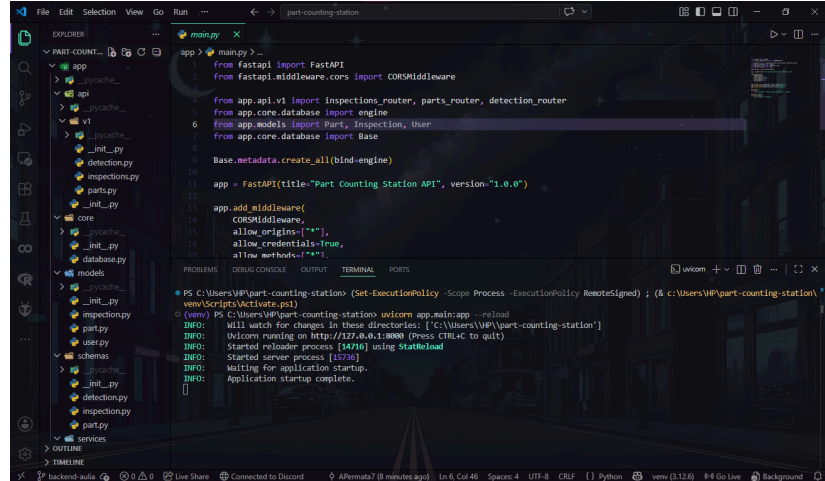
Algorithm: ☒ UNDEFINED ☐ MERGE ☐ TEMPTABLE

Check option for updates: ☒ None ☐ CASCADED ☐ LOCAL

```
1 select 'i'.inspection_id AS 'inspection_id', 'i'.inspection_time AS 'inspection_time', 'p'.part_name AS 'part_name', 'i'.operator_username AS
```

Name	Rows	Size	Created	Updated	Engine	Comment	Type
inspections	70	48.0 KiB	2026-04-22 23:08:11		InnoDB		Table
inspection_det...			2026-04-23 06:59:35			VIEW	View
part	6	32.0 KiB	2026-04-19 08:35:27		InnoDB		Table
users	0	32.0 KiB	2026-04-19 08:35:54		InnoDB		Table

API Backend (FastAPI)



The screenshot shows the VS Code editor with the `main.py` file open. The file contains the following code:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api.v1 import inspections_router, parts_router, detection_router
from app.core.database import engine
from app.models import Part, Inspection, User
from app.core.database import Base

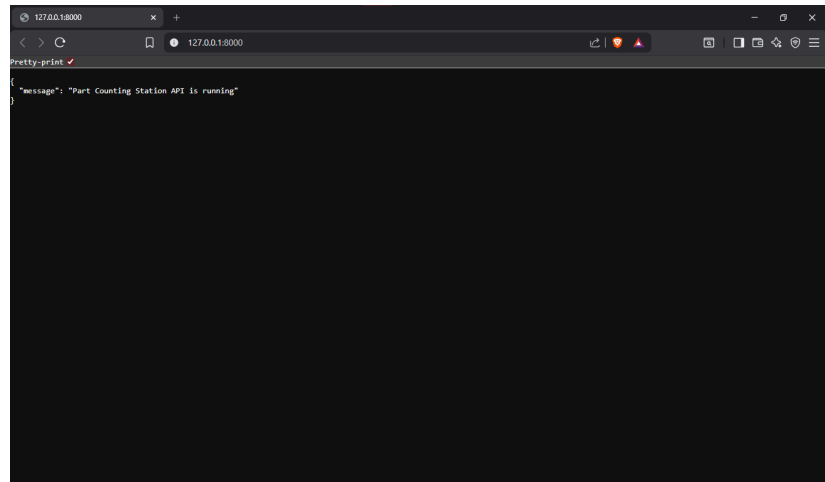
Base.metadata.create_all(bind=engine)

app = FastAPI(title="Part Counting Station API", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

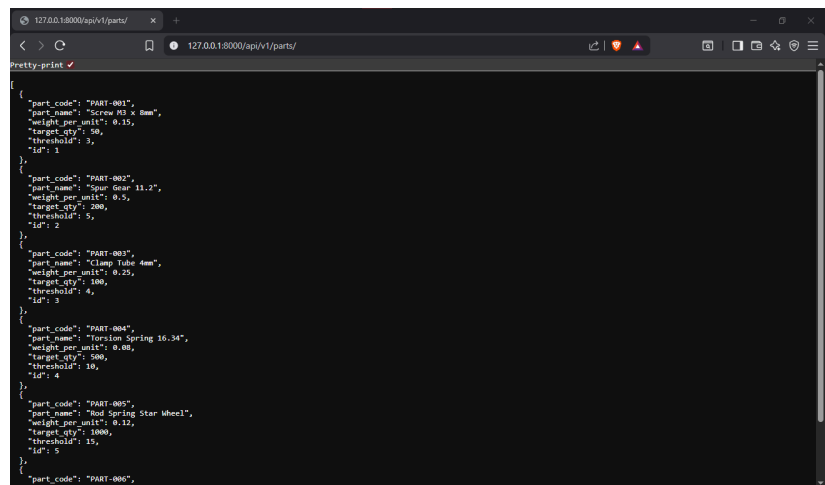
The terminal output shows the following messages:

```
PS C:\Users\VP\part-counting-station> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\VP\part-counting-station\venv\Scripts\activate.ps1)
(vmm) PS C:\Users\VP\part-counting-station> uvicorn app:mainapp --reload
INFO: Will watch for changes in these directories: ['C:\Users\VP\part-counting-station']
INFO: Uvicorn running on http://127.0.0.1:8000 (press CTRL+C to quit)
INFO: Started reload process (14716) using statreload
INFO: Started server process (15736)
INFO: Waiting for application startup.
INFO: Application startup complete.
```



The screenshot shows a web browser window with the URL `127.0.0.1:8000`. The response is a JSON object:

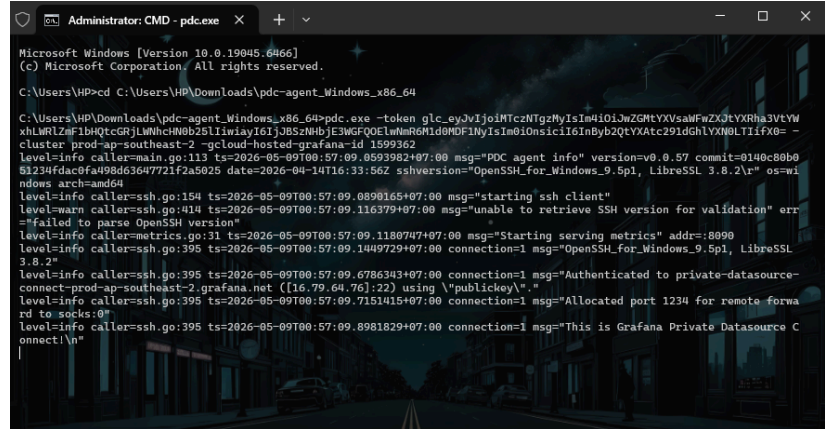
```
{
  "message": "Part Counting Station API is running"
}
```



The screenshot shows a web browser window with the URL `127.0.0.1:8000/api/v1/parts/`. The response is a JSON array of part objects:

```
[
  {
    "part_code": "PART-001",
    "part_name": "Screw 10 x 4mm",
    "weight_per_unit": 0.15,
    "target_qty": 50,
    "threshold": 3,
    "id": 1
  },
  {
    "part_code": "PART-002",
    "part_name": "Sprocket 11.2",
    "weight_per_unit": 0.5,
    "target_qty": 200,
    "threshold": 5,
    "id": 2
  },
  {
    "part_code": "PART-003",
    "part_name": "Clamp Tube 4mm",
    "weight_per_unit": 0.25,
    "target_qty": 100,
    "threshold": 4,
    "id": 3
  },
  {
    "part_code": "PART-004",
    "part_name": "Tension Spring 16.34",
    "weight_per_unit": 0.08,
    "target_qty": 500,
    "threshold": 10,
    "id": 4
  },
  {
    "part_code": "PART-005",
    "part_name": "Mod Spring Star Wheel",
    "weight_per_unit": 0.12,
    "target_qty": 1000,
    "threshold": 15,
    "id": 5
  },
  {
    "part_code": "PART-006",
    "part_name": "Mod Spring Star Wheel",
    "weight_per_unit": 0.12,
    "target_qty": 1000,
    "threshold": 15,
    "id": 5
  }
]
```

Integrasi Grafana Cloud via PDC Agent

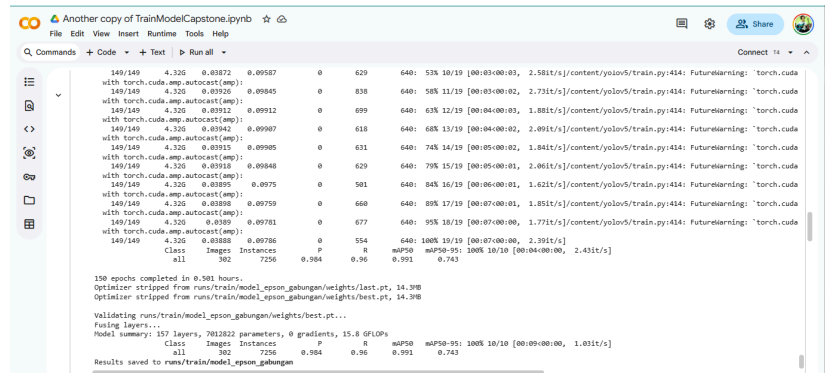


```
Administrator: CMD - pdc.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>cd C:\Users\HP\Downloads\pd-agent_Windows_x86_64

C:\Users\HP\Downloads\pd-agent_Windows_x86_64>pd-agent.exe -token glc_eyJvIjoImTczMTgzMyIsIm4iOiJwZGZMcVYsaWZlYXN3YXRha3VtYV
xhLWRLZmF1bHQtYGRjRjLMWmhcN0B2S1liiWiy161j3BSzN0bHjE3WGFQOEwNmRGM1d0MDF1NyIsIm8iOiNscic16InbYb2QlYXAtc291dGhVYXN0LTl1fX0= -
cluster prod-ap-southeast-2 -gcloud-hosted-grafana-id 1699362
level=info caller=main.go:113 ts=2026-05-09T00:57:09.08929324+07:00 msg="PDC agent info" version=v0.0.57 commit=0140c88b9
81234fdac8fa498d63647721f2a5025 date=2026-04-14T16:33:56Z sshversion="OpenSSH_for_Windows_9.5p1, LibreSSL 3.8.2" os=wi
ndows arch=amd64
level=info caller=ssh.go:154 ts=2026-05-09T00:57:09.0890165+07:00 msg="starting ssh client"
level=warn caller=ssh.go:414 ts=2026-05-09T00:57:09.116379+07:00 msg="unable to retrieve SSH version for validation" err
or=failed to parse OpenSSH version"
level=info caller=metrics.go:31 ts=2026-05-09T00:57:09.1180747+07:00 msg="Starting serving metrics" addr=:8090
level=info caller=ssh.go:395 ts=2026-05-09T00:57:09.1449729+07:00 connection=1 msg="OpenSSH_for_Windows_9.5p1, LibreSSL
3.8.2"
level=info caller=ssh.go:395 ts=2026-05-09T00:57:09.6786343+07:00 connection=1 msg="Authenticated to private-datasource-
connect-prod-ap-southeast-2.grafana.net ([16.79.64.76]:22) using \publickey\".
level=info caller=ssh.go:395 ts=2026-05-09T00:57:09.7151415+07:00 connection=1 msg="Allocated port 1234 for remote forwa
rd to socks:0"
level=info caller=ssh.go:395 ts=2026-05-09T00:57:09.8981829+07:00 connection=1 msg="This is Grafana Private Datasource C
onnect!\n"
```

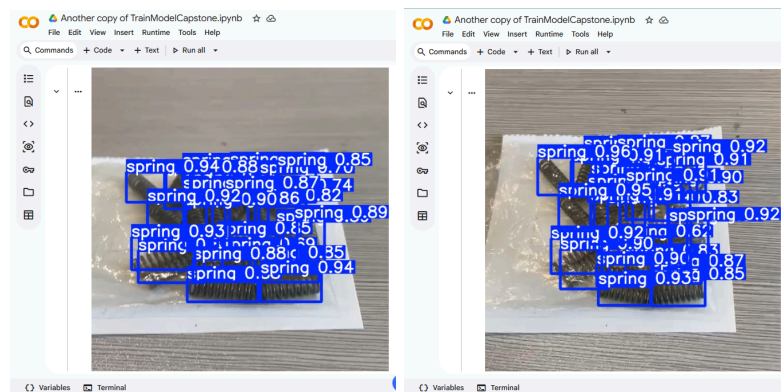
Model Training using YOLOv5



```
Another copy of TrainModelCapstone.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text + Run all
Connect 14
149/149 4.326 0.03872 0.09587 0 629 640: 53% 10/19 [00:03:00-03, 2.581t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03920 0.09845 0 838 640: 58% 11/19 [00:03:00-02, 2.731t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03912 0.09912 0 899 640: 63% 12/19 [00:04:00-03, 1.881t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03942 0.09907 0 618 640: 68% 13/19 [00:04:00-02, 2.891t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03915 0.09905 0 631 640: 74% 14/19 [00:05:00-02, 1.841t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03919 0.09948 0 629 640: 79% 15/19 [00:05:00-01, 2.861t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03895 0.0975 0 501 640: 84% 16/19 [00:06:00-01, 1.621t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03888 0.09759 0 668 640: 89% 17/19 [00:07:00-01, 1.851t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.0389 0.09781 0 677 640: 95% 18/19 [00:07:00-00, 1.771t/s]/content/yolo5/train.py:414: FutureWarning: `torch.cuda
with torch.cuda.amp.autocast(enabled=True):
149/149 4.326 0.03888 0.09786 0 554 640: 100% 19/19 [00:07:00-00, 2.391t/s]
Class Images Instances P R mAP50 mAP50-95 100% 10/19 [00:09:00-00, 1.031t/s]
all 362 7250 0.964 0.96 0.991 0.743
150 epochs completed in 0.580 hours.
Optimizer stripped from runs/train/model_000000/weights/last.pt, 14.34M
Optimizer stripped from runs/train/model_000000/weights/best.pt, 14.34M
Validating runs/train/model_000000/weights/best.pt...
Fusing layers...
Model summary: 157 layers, 701282 parameters, 0 gradients, 15.8 GFLOPs
Class Images Instances P R mAP50 mAP50-95 100% 10/19 [00:09:00-00, 1.031t/s]
all 362 7250 0.964 0.96 0.991 0.743
Results saved to runs/train/model_000000
```

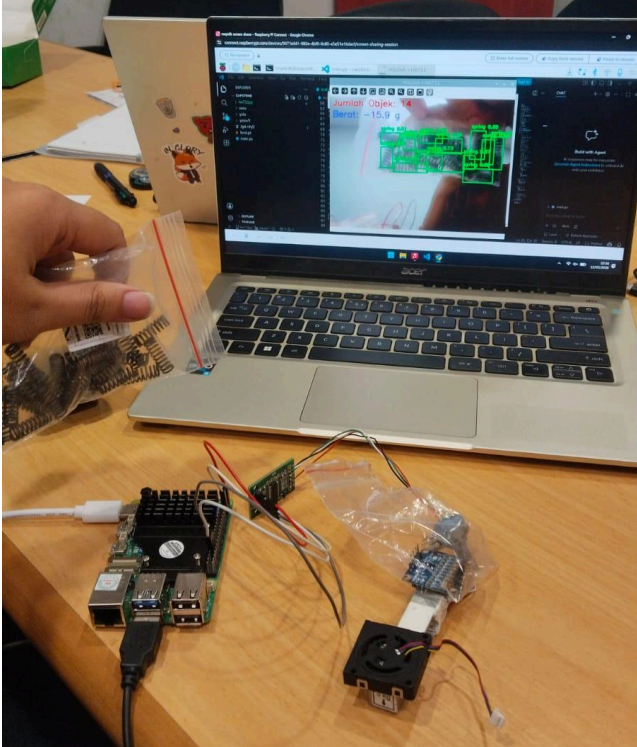
Deskripsi: Tangkapan layar (*log console*) proses pelatihan model YOLOv5 di Google Colab. Log mengonfirmasi proses optimasi bobot model yang berjalan mulus hingga 150 *epochs*, di mana metrik *loss* secara konsisten menurun dan konvergensi tercapai dengan baik.

Model Testing using YOLOv5



Deskripsi: Tangkapan layar hasil inferensi (*testing*) model YOLOv5 pada *test set*. Bukti ini menunjukkan bahwa model tidak hanya berhasil menggambar *bounding box* pada objek yang bertumpuk (*overlap*), tetapi skrip Python kustom juga

	berhasil mengekstrak data tersebut menjadi teks kalkulasi total <i>spring</i> secara otomatis di layar.											
Model Training using SSD MobileNet V2 (for comparison)	DetectionBoxe... /mAP@.50IOU <table><tr><th>Run</th><th>Smoothed Value</th><th>Step</th><th>Time</th><th>Relative</th></tr><tr><td>eval</td><td>0.8693</td><td>0.8693</td><td>5,000</td><td>5/8/26, 1:57 PM</td><td>0</td></tr></table> Hasil Deteksi	Run	Smoothed Value	Step	Time	Relative	eval	0.8693	0.8693	5,000	5/8/26, 1:57 PM	0
Run	Smoothed Value	Step	Time	Relative								
eval	0.8693	0.8693	5,000	5/8/26, 1:57 PM	0							

	Deskripsi: Tangkapan layar proses pelatihan model komparasi SSD MobileNet V2 menggunakan <i>framework</i> Google MediaPipe. Bukti ini menunjukkan keberhasilan penyesuaian lingkungan (<i>environment patching</i>) untuk melatih arsitektur ringan yang menghasilkan output berformat .tflite, khusus dipersiapkan untuk evaluasi performa di <i>edge device</i>.
Integrasi Computer Vision dengan Load Cell di Raspberry pi	

E.3 Kendala Implementasi

Kendala	Solusi
<i>Dashboard monitoring</i> belum memiliki data aktual produksi pada tahap awal pengembangan.	Menggunakan database <i>dummy</i> yang dihubungkan ke Grafana Cloud untuk melakukan simulasi visualisasi data dan pengembangan dashboard KPI sementara.
Database MySQL masih menggunakan data <i>dummy</i> (belum terintegrasi langsung dengan sensor).	Data <i>dummy</i> dimasukkan melalui query SQL langsung ke tabel <i>inspections</i> . Setelah integrasi dengan Raspberry Pi selesai, data akan berasal dari inspeksi nyata.
Autentikasi dan role-based access (JWT,	Saat ini endpoint API masih terbuka tanpa

RBAC) belum diimplementasikan sepenuhnya.	otentikasi. Rencananya akan ditambahkan setelah fitur inti selesai, menggunakan python-jose dan passlib.
Fitur export laporan (CSV/Excel) belum tersedia.	Akan ditambahkan endpoint /api/v1/export dengan parameter filter, menggunakan library pandas atau csv bawaan Python.
Backend saat ini hanya dapat diakses dalam satu jaringan lokal; belum bisa diakses secara publik oleh tim dari lokasi berbeda.	Sedang mengevaluasi solusi tunneling (ngrok, localhost.run) atau jaringan virtual (Tailscale/ZeroTier) untuk akses jarak jauh tanpa mengorbankan keamanan.
Inkompatibilitas <i>Environment & Library Usang</i> Terjadi <i>dependency conflict</i> saat melatih model komparasi. Pembaruan sistem bawaan Google Colab (ke Python 3.12) menghapus dukungan fitur lawas (<i>estimator</i>) pada TensorFlow Object Detection API, menyebabkan <i>error</i> sistematis (ModuleNotFoundError).	Pivoting Framework & Isolasi <i>Environment</i> Mengubah strategi dengan menggunakan <i>framework</i> Google MediaPipe Model Maker yang lebih modern. Selain itu, mengeksekusi skrip secara manual melalui <i>Command Line Interface</i> (CLI) menggunakan mesin Python 3.10 yang diinstal terpisah secara paksa (<i>force install</i>) agar terhindar dari bentrok versi sistem utama.
Limitasi <i>Resource GPU & Durasi Training</i> Proses pelatihan model YOLOv5 (150 <i>epochs</i>) membutuhkan beban komputasi tinggi. Sesi <i>cloud</i> sering mengalami <i>timeout</i> , dan akun mengalami pembatasan (<i>usage limit</i>) GPU gratis dari Google Colab di tengah proses <i>testing</i> .	Rotasi <i>Runtime & Checkpointing</i> Memanfaatkan sistem <i>checkpointing</i> (last.pt) pada YOLO untuk menyambung <i>training</i> yang terputus, serta melakukan manajemen rotasi akun/sesi secara dinamis untuk mengembalikan akses akselerator GPU T4 saat limitasi terjadi.
Konflik Standar Struktur Dataset Saat melakukan penggabungan dataset (Nadhif & Zaka dari project roboflow yang berbeda) dan pemindahan <i>framework</i> , terjadi <i>error</i> pembacaan data (num_shards=0) karena perbedaan standar penamaan <i>folder</i> bawaan Pascal VOC (JPEGImages vs images).	Automasi Pra-pemrosesan Data Membangun skrip Python automasi untuk membedah (parsing), merombak struktur folder, dan memperbarui nama path di dalam seluruh file anotasi XML secara massal tanpa perlu mengunduh ulang dataset dari Roboflow.

Limitasi <i>Edge Resource Processing</i> Saat melakukan integrasi antara computer vision dan load cell terdapat kendala pada pemrosesan di Raspberry Pi, dimana terjadi delay saat pendeteksian objek.	Optimasi <i>Computer Vision</i> Pengoptimasian <i>computer vision</i> dengan menurunkan resolusi dari kamera
--	--

F. Pengujian Sederhana

F.1 Rencana Pengujian

Pengujian sistem difokuskan pada fitur utama yang mendukung proses verifikasi part dan visualisasi monitoring pada dashboard KPI. Pengujian dilakukan untuk memastikan setiap modul berjalan sesuai kebutuhan fungsional sistem.

Fitur Utama	Skenario Pengujian	Tujuan Pengujian	Hasil yang Diharapkan
Dashboard KPI	Dashboard menerima dan menampilkan data inspeksi dari database secara real-time.	Memastikan fitur monitoring dan visualisasi KPI berjalan dengan baik.	Dashboard berhasil menampilkan jumlah inspeksi, defect rate, jumlah OK/NG, dan tabel detail inspeksi secara real-time.
Riwayat dan Audit Trail	Pengguna mengakses histori inspeksi pada dashboard atau database.	Memastikan data inspeksi tersimpan dan dapat ditelusuri kembali.	Seluruh data histori inspeksi dapat ditampilkan sesuai data yang tersimpan pada database.
Sensor Fusion (Endpoint /detection/inspect)	Mengirim request POST dengan part_id, operator_username, n_cv, weight_gram ke API.	Memastikan logika perhitungan difference dan penentuan status OK/NG berjalan benar.	API mengembalikan respons JSON berisi status yang sesuai (OK jika difference $\leq$ threshold, NG jika lebih).
Integrasi PDC Agent	Memeriksa status koneksi PDC Agent di halaman Grafana Cloud.	Memastikan database lokal dapat diakses oleh Grafana Cloud secara aman.	Status agent menunjukkan 1 agents connected dan dashboard dapat menampilkan data terbaru.

Evaluasi Deteksi YOLOv5 (Kondisi Terkontrol)	Menginferensi sekumpulan gambar <i>test set</i> berisi <i>part spring</i> hitam di dalam plastik bening di atas permukaan meja.	Memvalidasi kemampuan dasar model dalam mendeteksi dan menghitung tumpukan <i>part</i> pada kondisi pencahayaan dan latar belakang yang sesuai dengan fase pelatihan.	Model berhasil menggambar <i>bounding box</i> yang presisi dan skrip mengekstrak jumlah <i>part</i> dengan akurasi (mAP50) sangat tinggi (>90%).
Uji Ketahanan Visual (<i>Robustness Test</i>)	Menginferensi citra <i>part spring</i> dengan variasi ekstrem: warna objek berbeda (merah) dan berlatar belakang kasur berpola biru.	Menguji kemampuan generalisasi algoritma AI terhadap perubahan drastis pada lingkungan (<i>background</i>) dan fitur visual objek.	Model seharusnya tetap dapat mengenali pola geometris <i>spring</i> secara utuh meskipun terdapat <i>noise</i> visual dari latar belakang.
Komparasi Arsitektur Model Alternatif	Mengeksekusi pelatihan dan pengujian metrik menggunakan arsitektur SSD MobileNet V2 pada dataset yang identik.	Mencari basis data pembandingan (<i>baseline</i>) untuk membuktikan justifikasi mengapa pendekatan YOLOv5 wajib dipertahankan untuk objek <i>overlap</i> mikro.	Sistem menghasilkan skor mAP50 komparasi yang dapat dianalisis kelemahan dan kelebihanannya.
Uji Inferensi Kamera <i>Real-Time</i>	Menghubungkan modul script perhitungan YOLOv5 (<i>detect.py</i>) langsung ke umpan video dari Webcam/Kamera PC.	Mengukur responsivitas, latensi (<i>delay</i>), dan nilai FPS (<i>Frame Per Second</i>) dari algoritma deteksi secara aktual.	Sistem mampu membaca <i>frame</i> kamera dan mencetak angka perhitungan <i>spring</i> secara dinamis tanpa jeda berarti.
Integrasi Sensor & <i>Sensor Fusion</i>	Menginput <i>quantity_expected</i> , meletakkan objek sampel, dan memicu fungsi <i>run_inspection()</i> .	Validasi data <i>payload</i> JSON yang dikirim ke <i>backend</i> menggunakan log konsol Python.	Sistem berhasil menggenerasi <i>inspection_id</i> unik, menangkap <i>quantity_cv</i> dan <i>quantity_loadcell</i> , menghitung <i>discrepancy</i> , serta

			menentukan status OK/NG secara akurat.
Performa <i>Throughput</i> Visual (FPS)	Menjalankan <i>video stream</i> dari USB Webcam Full HD bersamaan dengan proses inferensi model YOLOv5 lokal di Raspberry Pi 5.	Mengukur jumlah <i>frame</i> per detik menggunakan fungsi penanda waktu OpenCV (cv2.getTickCount).	Aliran video tetap berjalan lancar dengan <i>throughput</i> optimal (target 10 FPS) tanpa mengalami <i>stuttering</i> atau <i>freeze</i> pada sistem <i>edge</i> .

F.2 Hasil Pengujian

Fitur	Skenario	Hasil	Status
Evaluasi Deteksi YOLOv5 (Kondisi Terkontrol)	Inferensi citra <i>spring</i> hitam dalam plastik.	Skrip deteksi berhasil berjalan sempurna. Model mendeteksi <i>bounding box</i> objek <i>overlap</i> secara presisi dan menghasilkan kalkulasi <i>part</i> yang akurat dengan skor evaluasi metrik mAP50 mencapai 99.1% .	Berhasil
Uji Ketahanan Visual (<i>Robustness Test</i>)	Inferensi citra <i>spring</i> merah di atas kasur biru.	Deteksi gagal secara visual. Model kehilangan kemampuan mendeteksi objek (<i>undercounting</i> parah) karena kebingungan membedakan fitur <i>spring</i> dengan pola <i>background</i> kasur.	Gagal
Komparasi Arsitektur Model Alternatif	Inferensi model komparasi SSD MobileNet V2.	Proses komparasi berhasil dieksekusi, namun performa deteksi hancur secara fundamental. Model hanya mencetak mAP50 sebesar 86.9% , membuktikan YOLOv5 masi lebih unggul.	Berhasil Dieksekusi
Uji Inferensi Kamera Kamera <i>Real-Time</i>	Eksekusi input <i>Webcam</i> melalui modul skrip Python.	Pengujian belum dapat dilakukan pada lingkungan <i>cloud</i> (Google Colab) dan masih menunggu tahap pemindahan (<i>deployment</i>) ke perangkat <i>edge device</i> (Raspberry Pi).	Belum Dilakukan
Integrasi Dashboard Grafana Cloud	Dashboard menerima dan menampilkan data inspeksi dari database.	Dashboard KPI berhasil diintegrasikan dengan database <i>dummy</i> menggunakan Grafana Cloud untuk menampilkan visualisasi monitoring inspeksi secara real-time.	Berhasil Dieksekusi

Visualisasi KPI dan Monitoring	Dashboard menampilkan data inspeksi dari database.	Dashboard berhasil menampilkan panel visualisasi seperti total inspeksi, status OK/NG, <i>defect rate</i> , dan histori inspeksi menggunakan data <i>dummy</i> dari database.	Berhasil Dieksekusi
Integrasi Sensor & <i>Sensor Fusion</i>	Operator menginput <i>quantity_expected</i> , meletakkan sampel komponen, dan memicu fungsi inspeksi untuk memproses data <i>quantity_cv</i> dan <i>quantity_loadcell</i> secara bersamaan.	Sistem berhasil mengeksekusi integrasi secara sinkron. Data dari kedua sensor diproses secara komparatif untuk menghasilkan nilai <i>discrepancy</i> dan keputusan status (OK/NG). Hasil akhir dibungkus dalam <i>payload</i> JSON lengkap dengan <i>inspection_id</i> unik dan sukses ditransmisikan ke Backend API.	Berhasil
Performa Throughput Visual (FPS)	Menghubungkan webcam USB pada stasiun kerja dan menjalankan loop inferensi model YOLOv5 + SAHI secara kontinu lokal di Raspberry Pi 5.	Aliran video mengalami <i>stuttering</i> intensif saat proses inferensi aktif mendeteksi objek mikro yang padat. Pengukuran <i>frame rate</i> menggunakan fungsi penanda waktu OpenCV mencatat <i>throughput</i> konstan hanya berada pada kisaran 1 - 2 FPS.	Berhasil Dieksekusi

F.3 Ringkasan

F.3.1 Arsitektur Sistem AI Module

Ringkasan Model Computer Vision

- **Sistem Berjalan / Tidak:** Secara fundamental, modul *Computer Vision* dan otomatisasi penghitungannya **BERJALAN** dengan sangat memuaskan. Tingkat akurasi utama model YOLOv5 (**99.1%**) pada kondisi industrial yang terkontrol telah melampaui target awal proyek.
- **Bug Utama:** Model deteksi saat ini mengalami indikasi kuat terkena masalah ***Overfitting Lingkungan***. Model terlalu menghafal tekstur latar belakang dan warna objek pada *training set* awal (hitam dan meja kerja), sehingga menjadi "buta objek" (*blind spot*) ketika dihadapkan pada objek berwarna merah dengan latar belakang kasur yang ramai

(noisy background). **Solusi ke depan:** Tim AI (TIF) akan memperkaya dataset dengan teknik *Image Augmentation* (variasi warna, kecerahan, dan rotasi *background* acak).

F.3.2 Ringkasan UI Dashboard Monitoring

- **Sistem Berjalan / Tidak:** Secara umum, modul UI *Dashboard Monitoring* **BERJALAN** dengan baik pada tahap pengembangan awal. *Dashboard* KPI berhasil divisualisasikan menggunakan Grafana Cloud dan telah terintegrasi dengan *database dummy* untuk menampilkan informasi seperti total inspeksi, status OK/NG, *defect rate*, dan histori inspeksi dalam bentuk visualisasi interaktif.
- **Bug Utama:** Integrasi real time dengan backend dan Raspberry Pi masih dalam tahap pengembangan sehingga **sinkronisasi data *live monitoring* belum sepenuhnya stabil**. *Dashboard* saat ini masih menggunakan data *dummy* sehingga belum sepenuhnya merepresentasikan kondisi inspeksi aktual dari sistem *Part Counting Station*.

G. Evaluasi Sistem

G.1 Kesesuaian Dengan Kebutuhan

Secara keseluruhan, sistem *Part Counting Station* yang diimplementasikan telah menunjukkan kesesuaian yang tinggi terhadap kebutuhan fungsional dan non-fungsional yang ditetapkan oleh mitra PT Indonesia Epson Industry. Mekanisme *Triple-Check Verification* berhasil menyelaraskan input target kuantitas dengan hasil deteksi visual dan berat secara *real-time*. Meskipun pengujian performa visual pada stasiun kerja mengalami *computational bottleneck* lokal yang membatasi *throughput* hingga 1–2 FPS, sistem tetap memenuhi syarat non-fungsional utama karena total waktu respons (*delay execution*) per sesi pemeriksaan tetap berada di bawah batas toleransi 3 detik. Sistem juga berhasil melampaui target akurasi klasifikasi awal kelompok sebesar 90 pada kondisi industrial terkontrol.

G.2 Fitur Yang Sudah Terpenuhi

1. Dataset Collection & Labeling: Pengumpulan dan anotasi data komponen *spring* menggunakan platform Roboflow telah rampung 100%.
2. Model Training YOLOv5: Pelatihan model inti menggunakan arsitektur YOLOv5 berhasil mencapai nilai evaluasi mAP50 superior sebesar 99.1% dalam mendeteksi tumpukan part.
3. Automated Data Extraction Script: Pengembangan skrip Python kustom untuk mengekstrak data koordinat dan menghitung kuantitas numerik secara otomatis telah sukses diimplementasikan.
4. Database & Backend API Gateway: Pembuatan database MySQL serta penyediaan *endpoint* RESTful menggunakan FastAPI telah berhasil dihubungkan.
5. Integrasi Grafana via PDC Agent: Jembatan *tunnel* SSH satu arah berhasil menghubungkan database lokal ke Grafana Cloud secara aman tanpa membuka port *firewall*.

6. Integrasi Hardware Edge: Sinkronisasi fungsional antara USB Webcam dan sensor *load cell* (HX711) pada perangkat Raspberry Pi 5 telah berjalan dengan baik.
7. Pembuatan dan penerapan query DDL serta DML secara langsung pada panel dashboard Grafana telah berhasil dilakukan untuk kebutuhan visualisasi dan pengolahan data monitoring.
8. Sistem monitoring dashboard mendukung fitur filter data berdasarkan waktu untuk mempermudah proses analisis data.

G.3 Fitur Yang Belum Terpenuhi

1. Autentikasi & Otorisasi Penuh (JWT & RBAC): *Endpoint* API saat ini masih terbuka tanpa enkripsi dan pembatasan peran (*role access*) secara menyeluruh.
2. Fitur Export Reporting: Penyediaan fungsi ekspor data historis inspeksi ke dalam format dokumen CSV atau Excel masih dalam tahap antrian pengembangan.
3. Akses Backend Publik: Aksesibilitas server *backend* utama masih terbatas pada area jaringan lokal (*local network*) yang sama.

H. Kesimpulan

H.1 Progres Saat Ini

Proyek pengembangan *Part Counting Station Terintegrasi* saat ini telah mencapai fase implementasi *end-to-end* yang fungsional. Seluruh alur data dari pembacaan sensor fisik di perangkat *edge* lokal hingga pengiriman *payload* menuju database dan visualisasi indikator performa utama (KPI) pada *dashboard* Grafana Cloud telah berhasil terhubung dengan baik.

H.2 Hal Yang Sudah Berhasil

- Akurasi Deteksi Objek Mikro: Model YOLOv5 terbukti sangat superior dibandingkan dengan arsitektur SSD MobileNet V2 dalam menangani tumpukan komponen mikro padat yang saling bertumpuk (*overlap*).
- Sinkronisasi Multi-Sensor: Integrasi *Sensor Fusion* lokal menggunakan metode *Moving Average Filter* pada timbangan dan penangkapan citra via webcam berhasil menyajikan data komparasi jumlah yang presisi.
- Traceability Digital: Sistem otomatisasi audit trail digital berhasil mencatat setiap riwayat inspeksi lengkap dengan parameter unik serta penanda waktu ke database MySQL.

H.3 Rencana Berikutnya

- Pivoting Mode Inferensi Hardware: Mengubah skrip pemrosesan visual dari mode aliran berkelanjutan (*continuous streaming*) menjadi skema berbasis tangkapan gambar (*snapshot-based inference*) guna memitigasi isu rendahnya nilai FPS (1-2 FPS) dan menghindari risiko *overheating* pada CPU Raspberry Pi 5.

- Penyelesaian Fitur Keamanan dan Laporan: Mengimplementasikan pustaka python-jose dan passlib untuk mengunci keamanan data via JWT/RBAC , serta mengintegrasikan modul pandas untuk kebutuhan ekspor laporan operasional.
- Pengujian Data Aktual Produksi: Melakukan kalibrasi tingkat lanjut dan menguji ketahanan operasional sistem menggunakan komponen riil serta variasi pengkondisian pencahayaan langsung di lini produksi PT Indonesia Epson Industry.

I. Lampiran

Link Github:

<https://github.com/APermata7/part-counting-station>

Link Dataset:

<https://drive.google.com/drive/folders/1au3oqcRGL88W6C7xKzbQIDS1r2vRwvWu?usp=sharing>

J. Penilaian (Isi Dosen)

Aspek	Skor (1–4)
Desain Sistem	
Perencanaan	
Implementasi	
Pengujian	
Kesesuaian	